



All Tasks Scheduling Orders (hard)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
 - Code
- Time and Space Complexity

Problem Statement

There are 'N' tasks, labeled from '0' to 'N-1'. Each task can have some prerequisite tasks which need to be completed before it can be scheduled. Given the number of tasks and a list of prerequisite pairs, write a method to print all possible ordering of tasks meeting all prerequisites.

Example 1:

Input: Tasks=3, Prerequisites=[0, 1], [1, 2]
Output: [0, 1, 2]
Explanation: There is only possible ordering of the tasks.

Example 2:

Input: Tasks=4, Prerequisites=[3, 2], [3, 0], [2, 0], [2, 1]
Output:
1) [3, 2, 0, 1]
2) [3, 2, 1, 0]
Explanation: There are two possible orderings of the tasks meeting all prerequisites.

Example 3:

Input: Tasks=6, Prerequisites=[2, 5], [0, 5], [0, 4], [1, 4], [3, 2], [1, 3]
Output:
1) [0, 1, 4, 3, 2, 5]
2) [0, 1, 3, 4, 2, 5]
3) [0, 1, 3, 2, 4, 5]
4) [0, 1, 3, 2, 5, 4]
5) [1, 0, 3, 4, 2, 5]
6) [1, 0, 3, 2, 4, 5]
7) [1, 0, 3, 2, 5, 4]
8) [1, 0, 4, 3, 2, 5]
9) [1, 3, 0, 2, 4, 5]
10) [1, 3, 0, 2, 5, 4]
11) [1, 3, 0, 4, 2, 5]



12) [1, 3, 2, 0, 5, 4]
 13) [1, 3, 2, 0, 4, 5]

Try it yourself



Try solving this question here:

Java
 Python3
 JS
 C++

```

1  import java.util.*;
2
3  class AllTaskSchedulingOrders {
4      public static void printOrders(int tasks, int[][] prerequisites) {
5          // TODO: Write your code here
6      }
7
8      public static void main(String[] args) {
9          AllTaskSchedulingOrders.printOrders(3, new int[][] { new int[] { 0, 1 }, new int[] { 1, 2 } });
10         System.out.println();
11
12         AllTaskSchedulingOrders.printOrders(4,
13             new int[][] { new int[] { 3, 2 }, new int[] { 3, 0 }, new int[] { 2, 0 }, new int[] { 2, 1 } });
14         System.out.println();
15
16         AllTaskSchedulingOrders.printOrders(6, new int[][] { new int[] { 2, 5 }, new int[] { 0, 5 }, new int[] { 0, 4 },
17             new int[] { 1, 4 }, new int[] { 3, 2 }, new int[] { 1, 3 } });
18         System.out.println();
19     }
20 }
  
```

Run

Save

Reset



Solution

This problem is similar to [Tasks Scheduling Order](#), the only difference is that we need to find all the topological orderings of the tasks.

At any stage, if we have more than one source available and since we can choose any source, therefore, in this case, we will have multiple orderings of the tasks. We can use a recursive approach with **Backtracking** to consider all sources at any step.

Code

Here is what our algorithm will look like:

Java
 Python3
 C++
 JS

```

1  import java.util.*;
2
3  class AllTaskSchedulingOrders {
4      public static void printOrders(int tasks, int[][] prerequisites) {
5          List<Integer> sortedOrder = new ArrayList<>();
6          if (tasks <= 0)
7              return;
8
9          // a. Initialize the graph
10         HashMap<Integer, Integer> inDegree = new HashMap<>(); // count of incoming edges for every vertex
11         HashMap<Integer, List<Integer>> graph = new HashMap<>(); // adjacency list graph
12         for (int i = 0; i < prerequisites.length; i++) {
  
```

```

12     inDegree.put(i, 0);
13     graph.put(i, new ArrayList<Integer>());
14 }
15
16 // b. Build the graph
17 for (int i = 0; i < prerequisites.length; i++) {
18     int parent = prerequisites[i][0], child = prerequisites[i][1];
19     graph.get(parent).add(child); // put the child into it's parent's list
20     inDegree.put(child, inDegree.get(child) + 1); // increment child's inDegree
21 }
22
23 // c. Find all sources i.e., all vertices with 0 in-degrees
24 Queue<Integer> sources = new LinkedList<>();
25 for (Map.Entry<Integer, Integer> entry : inDegree.entrySet()) {
26     if (entry.getValue() == 0)
27         sources.add(entry.getKey());
28 }
29
30 printAllTopologicalSorts(graph, inDegree, sources, sortedOrder);
31 }
32
33 private static void printAllTopologicalSorts(HashMap<Integer, List<Integer>> graph,
34     HashMap<Integer, Integer> inDegree, Queue<Integer> sources, List<Integer> sortedOrder) {
35     if (!sources.isEmpty()) {
36         for (Integer vertex : sources) {
37             sortedOrder.add(vertex);
38             Queue<Integer> sourcesForNextCall = cloneQueue(sources);
39             // only remove the current source, all other sources should remain in the queue for the next call
40             sourcesForNextCall.remove(vertex);
41             List<Integer> children = graph.get(vertex); // get the node's children to decrement their in-degrees
42             for (int child : children) {
43                 inDegree.put(child, inDegree.get(child) - 1);
44                 if (inDegree.get(child) == 0)
45                     sourcesForNextCall.add(child); // save the new source for the next call
46             }
47         }
48
49         // recursive call to print other orderings from the remaining (and new) sources
50         printAllTopologicalSorts(graph, inDegree, sourcesForNextCall, sortedOrder);
51
52         // backtrack, remove the vertex from the sorted order and put all of its children back to consider
53         // the next source instead of the current vertex
54         sortedOrder.remove(vertex);
55         for (int child : children)
56             inDegree.put(child, inDegree.get(child) + 1);
57     }
58 }
59
60 // if sortedOrder doesn't contain all tasks, either we've a cyclic dependency between tasks, or
61 // we have not processed all the tasks in this recursive call
62 if (sortedOrder.size() == inDegree.size())
63     System.out.println(sortedOrder);
64 }
65
66 // makes a deep copy of the queue
67 private static Queue<Integer> cloneQueue(Queue<Integer> queue) {
68     Queue<Integer> clone = new LinkedList<>();
69     for (Integer num : queue)
70         clone.add(num);
71     return clone;
72 }
73
74 public static void main(String[] args) {
75     AllTaskSchedulingOrders.printOrders(3, new int[][] { new int[] { 0, 1 }, new int[] { 1, 2 } });
76     System.out.println();
77
78     AllTaskSchedulingOrders.printOrders(4,
79         new int[][] { new int[] { 3, 2 }, new int[] { 2, 0 }, new int[] { 2, 0 }, new int[] { 2, 1 } });

```

```
80     System.out.println();
81
82     AllTaskSchedulingOrders.printOrders(6, new int[][] { new int[] { 2, 5 }, new int[] { 0, 5 }, new int[] { 0, 4
83         new int[] { 1, 4 }, new int[] { 3, 2 }, new int[] { 1, 3 } });
84     System.out.println();
85 }
86 }
```

Run

Save

Reset



Time and Space Complexity

If we don't have any prerequisites, all combinations of the tasks can represent a topological ordering. As we know, that there can be $N!$ combinations for 'N' numbers, therefore the time and space complexity of our algorithm will be $O(V * E)$ where 'V' is the total number of tasks and 'E' is the total prerequisites. We need the 'E' part because in each recursive call, at max, we remove (and add back) all the edges.

[← Back](#)[✓ Mark As Completed](#)[Next →](#)

Tasks Scheduling Order (medium)

Alien Dictionary (hard)